

PROGRAMAÇÃO FUNCIONAL

FUNÇÕES RECURSIVAS

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

OBJETIVOS DA AULA

- Compreender polimorfismo em Haskell.
- Compreender os construtores de listas.
- Compreender as tuplas.
- Compreender as Definições Locais.

LISTAS

Estrutura de dados **fundamental** em programação funcional.

- Listas possuem dois padrões básicos
 - $[]$ denota a lista vazia.
 - $x : xs$ denota a lista com cabeça(primeiro elemento) x e cauda xs (demais elementos).

Chamei de x e xs , mas poderia ser qualquer nome.

LISTAS

Listas são tão comuns em Haskell que a linguagem possui sintaxe especial para representá-las.

Exemplo:

- $1 : (2 : (3 : []))$ pode ser escrita como $[1,2,3]$.

`:"` é o operador de inserção na lista

LISTAS

Exemplos:

- $1 : [5]$ -- Elemento 1 inserido na lista que contem o elemento 5
- $2 : (4 : [1, 6])$ -- Elemento 2 inserido na lista resultante de inserir o elemento 4 na lista contendo $[1, 6]$;

LISTAS

Como Haskell é uma linguagem **fortemente** tipadas, listas possuem **tipos**.

O tipo **[a]** denota listas cujos elementos são do tipo **a**.

- Listas são **polimórficas**.
- O **a** em **[a]** denota uma **variável de tipo**, que são universalmente quantificadas, ou seja é indiferente ao tipo.

LISTAS

Construtor de lista:

- $[] :: []$
- $(:) :: a \rightarrow [a] \rightarrow [a]$

LISTAS

Algumas funções sobre listas:

- **null** testa se uma lista é ou não vazia.
- **head** retorna o primeiro elemento de uma lista.
- **tail** retorna a cauda de uma lista.

head e **tail** resultam em erro se chamadas para a lista vazia.

LISTAS

Quais os tipos das funções **null**, **head** e **tail**?

- A função **null** verifica se uma lista é **vazia** ou não
 - $\text{null} :: [a] \rightarrow \text{Bool}$
- A função **head** retorna o primeiro **elemento** de uma **lista**.
 - $\text{head} :: [a] \rightarrow a$
- A função **tail** retorna a **cauda** de uma **lista**.
 - $\text{tail} :: [a] \rightarrow [a]$

LISTAS

Por que as assinaturas das funções **null**, **head** e **tail** utilizam variáveis de tipo **genéricas** (ex: **a**)?

- **Independência de Conteúdo:** Para verificar se uma lista está **vazia** ($\text{null} :: [a] \rightarrow \text{Bool}$), a função inspeciona estritamente a **estrutura do dado** (o **construtor []**). A natureza dos elementos armazenados é **irrelevante** para essa verificação.

LISTAS

Por que as assinaturas das funções **null**, **head** e **tail** utilizam variáveis de tipo **genéricas** (ex: **a**)?

- **Acesso Estrutural:** Para extrair o **primeiro** elemento ($\text{head} :: [a] \rightarrow a$) ou a **cauda** da lista ($\text{tail} :: [a] \rightarrow [a]$), a operação computacional **não requer conhecimento** prévio sobre o **tipo** concreto do dado sendo manipulado.

CASAMENTO DE PADRÃO EM LISTAS

O mecanismo de **casamento de padrão** permite a inspeção da **estrutura** de uma lista através de seus **construtores fundamentais**: a **lista vazia** (`[]`) e o operador **cons** (`:`).

CASAMENTO DE PADRÃO EM LISTAS

A extração de dados pode ser realizada de maneira **estática**, limitando-se a um **número finito de elementos**, sem a necessidade de **indução** ou **iteração**.

TUPLAS

- Haskell possui suporte a tuplas: pares, triplas, etc.
- O **tipo** de tuplas é (a,b) , em que **a** e **b** são tipos **quaisquer**.

TUPLAS

- Funções sobre tuplas.

1. $\text{first} :: (a,b) \rightarrow a$

2. $\text{first } (x, _) = x$

3.

4. $\text{second} :: (a,b) \rightarrow b$

5. $\text{second } (_, y) = y$

TUPLAS

- Exemplo: distância entre pontos
- $\text{dist} :: (\text{Float}, \text{Float}) \rightarrow (\text{Float}, \text{Float}) \rightarrow \text{Float}$
- $\text{dist } (x1, y1) (x2, y2)$
- $= \text{sqrt } ((x1 - x2)^2 + (y1 - y2)^2)$

TUPLAS

Diferença entre **tuplas** e **listas**:

- Tuplas podem possuir elementos de tipos diferentes.
- Listas são sempre homogêneas.

`(True , "abc", 2) :: (Bool, String, Int)`

CONDICIONAIS

Expressões **if** devem **obrigatoriamente** possuir o **else**.

- Tanto o **then** quanto o **else** devem possuir o **mesmo** tipo.

$\text{abs1} :: \text{Int} \rightarrow \text{Int}$

$\text{abs1} = \text{if } n < 0 \text{ then } (-1) * n \text{ else } n$

CONDICIONAIS

Outro exemplo:

1. `signal1 :: Int -> Int`

2. `signal1 n = if n < 0 then -1`

3. `else if n == 0 then 0`

4. `else 1`

GUARDAS

Maneira elegante de expressar condicionais

- **otherwise** é definido como **True** e pode ser usado como **padrão**.

1. `abs2 :: Int -> Int`

2. `abs2 n`

3. `| n < 0 = n * (-1)`

4. `| otherwise = n`

GUARDAS

Maneira elegante de expressar condicionais

- **otherwise** é definido como **True** e pode ser usado como **padrão**.

1. signal2 :: Int -> Int

2. signal2 n

3. | n < 0 = -1

4. | n == 0 = 0

5. | otherwise = 1

CONDICIONAIS

- que escolher, guardas ou condicionais?
 - Guardas favorem a leitura em relação a condicionais.

DEFINIÇÕES LOCAIS

Definições **let**

- Permite a definição de variáveis locais **antes** de seu uso.

DEFINIÇÕES LOCAIS

1. $\text{dist1} :: (\text{Float}, \text{Float}) \rightarrow (\text{Float}, \text{Float}) \rightarrow \text{Float}$
2. $\text{dist1 } (x1, y1) (x2, y2)$
3. $= \text{let}$
4. $dx = x1 - x2$
5. $dx2 = dx * dx$
6. $dy = y1 - y2$
7. $dy2 = dy * dy$
8. $\text{in } \text{sqrt } (dx2 + dy2)$

DEFINIÇÕES LOCAIS

Definições **where**

- Permite a definição de variáveis locais **depois** de seu uso.

DEFINIÇÕES LOCAIS

- $\text{dist2} :: (\text{Float}, \text{Float}) \rightarrow (\text{Float}, \text{Float}) \rightarrow \text{Float}$
- $\text{dist2 } (x1, y1) (x2, y2)$
- $= \text{sqrt } (dx2 + dy2)$
- where
- $dx = x1 - x2$
- $dx2 = dx * dx$
- $dy = y1 - y2$
- $dy2 = dy * dy$

DEFINIÇÕES LOCAIS

- Tanto **let** quanto **where** atribuem um **nome** a uma **expressão**.
- Benefícios:
 - Reduzir repetição de código
 - Documentação: atribuir nomes significativos a expressões.

DEFINIÇÕES LOCAIS

- Você pode ter um número qualquer definições locais a uma equação.
- Regra de **layout**: Definições devem começar em uma mesma **coluna**.

COMENTÁRIOS

Comentário de uma única linha:

1.-- Euler constant

2.e :: Float

3.e = 2.718

COMENTÁRIOS

Comentários de múltiplas linhas:

1. {-
2. Definindo uma lista vazia
3. -}
4. `empty :: [a]`
5. `empty = []`

CASAMENTO DE PADRÃO

$\text{fac} :: \text{Int} \rightarrow \text{Int}$

$\text{fac } 0 = 1$

$\text{fac } n = n * \text{fac } (n - 1)$

- Primeira equação executada quando o argumento é 0.
- Caso contrário, segunda equação é avaliada.

CASAMENTO DE PADRÃO

Exemplo: **replicate n x**

- Constrói uma lista com **n** cópias do valor **x**.
- Se **n** é **0**, retorne a **lista vazia**.
- Caso contrário, insira **x** à frente da **lista** contendo **n - 1** cópias de **x**.

CASAMENTO DE PADRÃO

- Definição de replicate.
 - Se um padrão não é usado, substitua-o por “_”

1. $\text{replicate} :: \text{Int} \rightarrow a \rightarrow [a]$

2. $\text{replicate } 0 _ = []$

3. $\text{replicate } n \ x = x : \text{replicate } (n - 1) \ x$

CASAMENTO DE PADRÃO

Definindo a conjunção ("and" lógico)

1. conj :: Bool -> Bool -> Bool
2. conj False False = False
3. conj False True = False
4. conj True False = False
5. conj True True = True

CASAMENTO DE PADRÃO

Padrões sobre listas:

1. $\text{null} :: [a] \rightarrow \text{Bool}$

2. $\text{null} [] = \text{True}$

3. $\text{null} (_ : _) = \text{False}$

CASAMENTO DE PADRÃO

Considere a seguinte definição

1. $\text{size } xs$
2. $| xs == [] = 0$
3. $| \text{otherwise} = 1 + \text{tail } xs$

CASAMENTO DE PADRÃO

Problemas com o código anterior:

- Nem todo **tipo** possui **igualdade implementada**, logo não há **garantia** que o código acima seja **aceito** para **todos** os tipos.
- **Casamento de padrão é compilado** para código mais **eficiente** que a **igualdade**, que depende de **sobrecarga**.

CASAMENTO DE PADRÃO

Uma definição usando casamento de padrão:

1. $\text{size} :: [a] \rightarrow \text{Int}$

2. $\text{size} [] = 0$

3. $\text{size} (_ : xs) = 1 + \text{size } xs$

EXERCÍCIOS

- Exercício 1: Comprimento da Lista
- Objetivo: Calcular o número de elementos de uma lista utilizando recursão e pattern matching.

```
contaElementos :: [a] -> Int  
contaElementos = undefined
```

EXERCÍCIOS

- Exercício 2: Soma Alternada
- Objetivo: Somar e subtrair alternadamente os elementos de uma lista numérica.
- Exemplo: $[1, 2, 3, 4] \rightarrow 1 - 2 + 3 - 4$.
- Dica: Utilize pattern matching avançado $(x:y:xs)$ para processar os pares.

```
somaAlternada :: Num a => [a] -> a  
somaAlternada = undefined
```

EXERCÍCIOS

- Exercício 3: Remoção de Duplicatas Consecutivas
- Objetivo: Remover elementos idênticos que estejam adjacentes na lista.
- Exemplo: $[1, 1, 2, 3, 3, 3, 4] \rightarrow [1, 2, 3, 4]$.
- Dica: Será necessário utilizar Guards em conjunto com a desestruturação.

```
removeConsecutivos :: Eq a => [a] -> [a]  
removeConsecutivos = undefined
```

EXERCÍCIOS

- Exercício 4: Intercalação de Estruturas (Zip)
- Objetivo: Mesclar duas listas independentes em uma **lista de tuplas**.
- A recursão deve ser interrompida assim que qualquer uma das listas alcançar o seu caso base (lista vazia).

$\text{mesclaListas} :: [a] \rightarrow [b] \rightarrow [(a, b)]$
`mesclaListas = undefined`

EXERCÍCIOS

- Exercício 5: Agrupamento em Pares
- Objetivo: Converter uma lista plana em uma lista de tuplas contendo pares adjacentes. Caso a lista original possua um número ímpar de elementos, o último elemento deve ser descartado.
- Ex: $[1, 2, 3, 4, 5] \rightarrow [(1,2), (3,4)]$.

`agrupaPares :: [a] -> [(a, a)]`

`agrupaPares = undefined`

DÚVIDAS???